The Detailed Breakdown: Direct Integration (1)

The core trick to this design is **separation of concerns**. We do not put the mutable state inside the Macaroon. Instead, we put a static *rule* inside the Macaroon, and pass the mutable state alongside it.

1. Minting (The Immutable Setup)
When the target service issues the token, it creates a standard Macaroon. To make it "stateful," the service adds a first-party caveat. However, instead of a simple static check (like `time < 12:00`), the caveat dictates a policy rule, such as `stateful_policy == access_only_created_wasm`. This caveat is permanently locked into the Macaroon's chained HMAC signature.

2. The Client Request (Token + Payload)
When the client app wants to access a resource, it must send two distinct pieces of data to the server:

- **The Macaroon:** The immutable credential proving authorization.
- **The State Payload:** The mutable log of past actions (e.g., a list of files the client has previously created), usually sent in the HTTP request header.

3. Dual Verification (The Server's Job)
When the target service receives the request, it performs two separate cryptographic checks:

- **Check the Macaroon:** The server validates the chained HMAC signature of the Macaroon to ensure the token and its caveats are authentic and untampered.
- **Check the State:** The server takes the client's State payload and computes an HMAC using a server-side secret key. It compares this result to a tiny 256-bit State HMAC it keeps in its own database. This guarantees the state payload is fresh and hasn't been forged by the client.

4. Execution and State Update
Once the server trusts both the Macaroon and the State payload, it evaluates the first-party caveat.

- It executes the WebAssembly (Wasm) attenuation policy, feeding it the current HTTP request and the verified State payload.
- If the Wasm policy returns `true`, the requested API action is executed.
- Finally, the server runs a Wasm "state updater" program to generate the *new* state, computes the new State HMAC to store in its database, and returns the new State payload to the client so the client has it for the next request.

The Detailed Breakdown: Fast Keygen Approach (2)

To understand this design, you have to think about "passing the baton" in a relay race, where each baton pass is cryptographically signed.

1. The Setup (Minting the Base Token)

- The Target Service generates the initial Macaroon.
- Instead of an HMAC root key, it generates an initial **Ed25519** key pair (Key Pair A).
- It signs the base token with Private Key A, attaches Public Key A to the token, and securely hands the token (and Private Key A) to the first client (Alice).

2. Adding a Caveat (The Delegation)
When Alice wants to restrict the token (e.g., add a "read-only" caveat) and pass it to Bob, she does the following:

- **Fast Keygen:** Alice generates a brand new, ephemeral Ed25519 key pair (Key Pair B). Because Ed25519 is highly optimized, this takes a fraction of a millisecond, completely bypassing the 54,000 µs RSA penalty the paper warned about.
- **The Signature:** Alice packages the new "read-only" caveat together with the new Public Key B. She signs this package using her current Private Key A.
- **The Handoff:** Alice appends the new caveat, Public Key B, and the new signature to the Macaroon. She then hands the Macaroon to Bob, along with the new Private Key B. Alice throws away Private Key A.

3. The Request and Verification
When Bob wants to use the token, he signs his HTTP request with his current private key (Private Key B) to prove he owns the end of the chain.

- **Public Verifiability:** Because all the signatures use public-key cryptography, the Target Service—or literally *anyone else* inspecting the token—can verify the entire chain of trust from start to finish.
- **The Check:** The verifier simply uses Public Key A to verify Alice's delegation to Public Key B, and then uses Public Key B to verify Bob's request.